UPN, PASIÓN POR TRANSFORMAR VIDAS

UNIDAD 1:

LISTAS, PILAS Y COLAS

SESIÓN 2:

Listas enlazadas simples

Dr. Eric GustavoCoronel Castillo
eric.coronel@upn.pe

MOTIVACIÓN

DESAFIO DEL DIA

NO ESPERES VER UN CAMBO, SI NO HACES NADA DIFERENTE.

MOTIVACIÓN

IDEA FUERZA

Solo al aprender y aplicar estructuras de datos (listas, pilas, colas, árboles, grafos) lograrán mejorar la eficiencia y la forma de resolver problemas

LOGRO DE LA UNIDAD 1

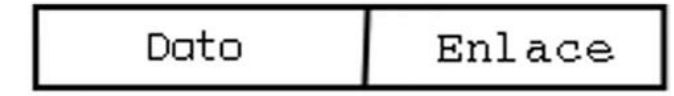
Al finalizar la unidad, el estudiante implementa algoritmos utilizando listas simples, listas dobles, listas circulares, pilas y colas, a través de control de versiones y el lenguaje C# con entorno gráfico, demostrando lógica y habilidad en la implementación de los algoritmos.

LOGRO DE LA SESIÓN

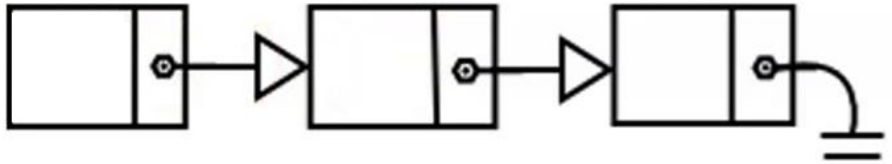
Al término de la sesión, el estudiante aprende algoritmos de listas enlazadas, buscar, mezclar y ordenar, usándolos con eficacia.

REFLEXIONA

1. ¿Qué es una lista enlazada?
2. ¿Qué es un nodo?
3. ¿Cómo se ordena, se busca y mezcla una lista simple?



Estructura de un nodo



Lista enlazada

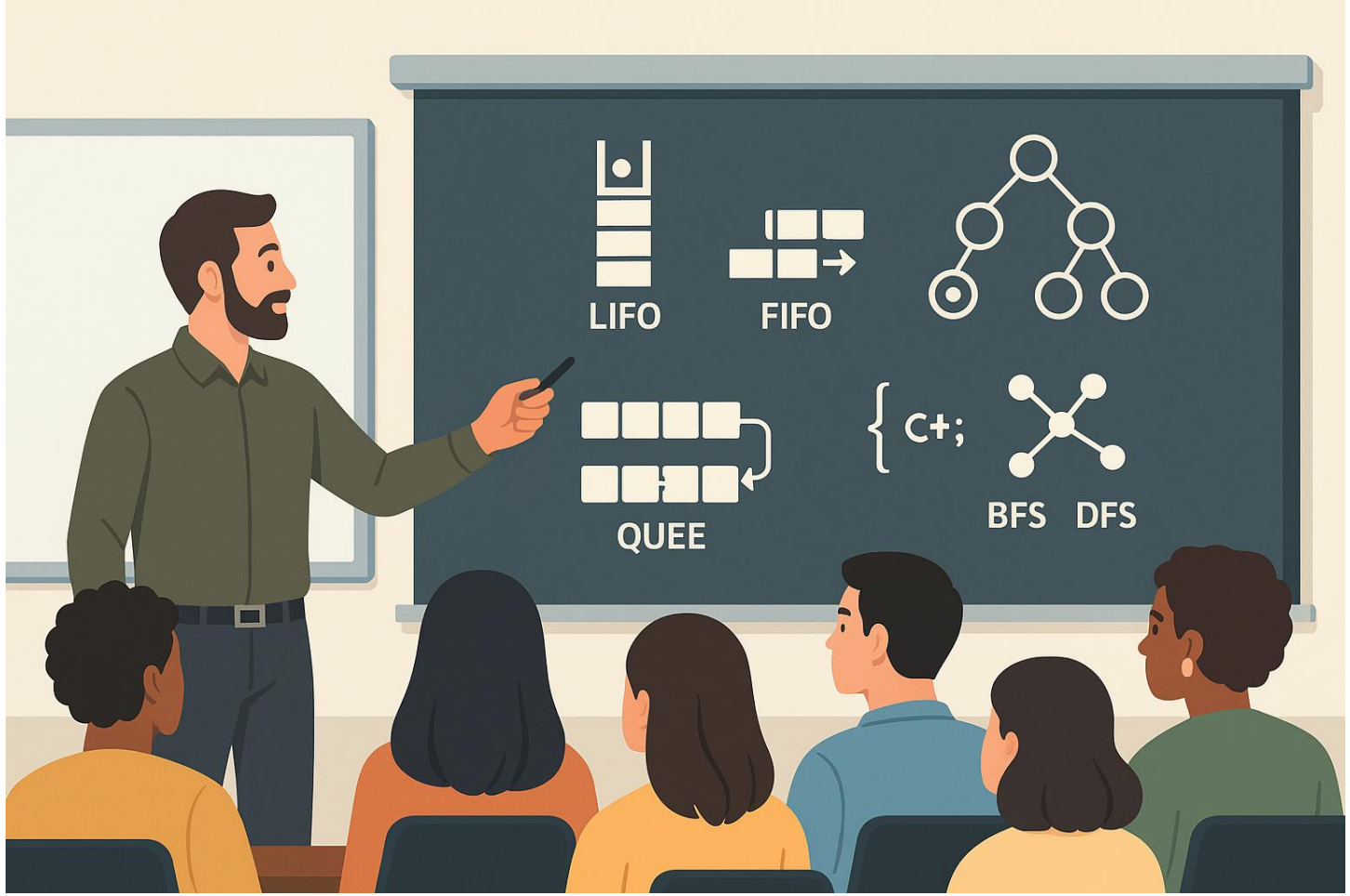
REFLEXIONA

En una lista enlazada, cada nodo no solo almacena información, también mantiene la conexión con el siguiente nodo.

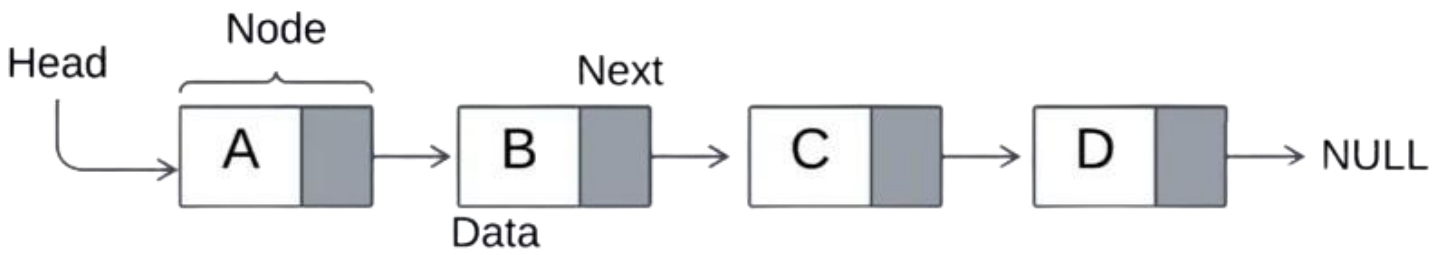
Esa relación entre dato y enlace es la base para comprender su recorrido, inserción y búsqueda.

```
class Nodo
{
        int dato;
        Nodo enlace;
        public Nodo(int t)
        {
        dato = t;
        enlace = null;
        }
}
```

DESARROLLO



INTRODUCCIÓN



- El principal beneficio de las listas enlazadas respecto a los arreglos convencionales es que el orden de los elementos enlazados puede ser diferente al orden de almacenamiento en memoria o el disco, permitiendo que el orden de recorrido de la lista sea diferente al de almacenamiento.
- Las listas enlazadas permiten inserciones y eliminaciones eficientes cuando se cuenta con la referencia al nodo o al nodo anterior, pero no permiten acceso aleatorio; para acceder a un elemento se requiere recorrer la lista.

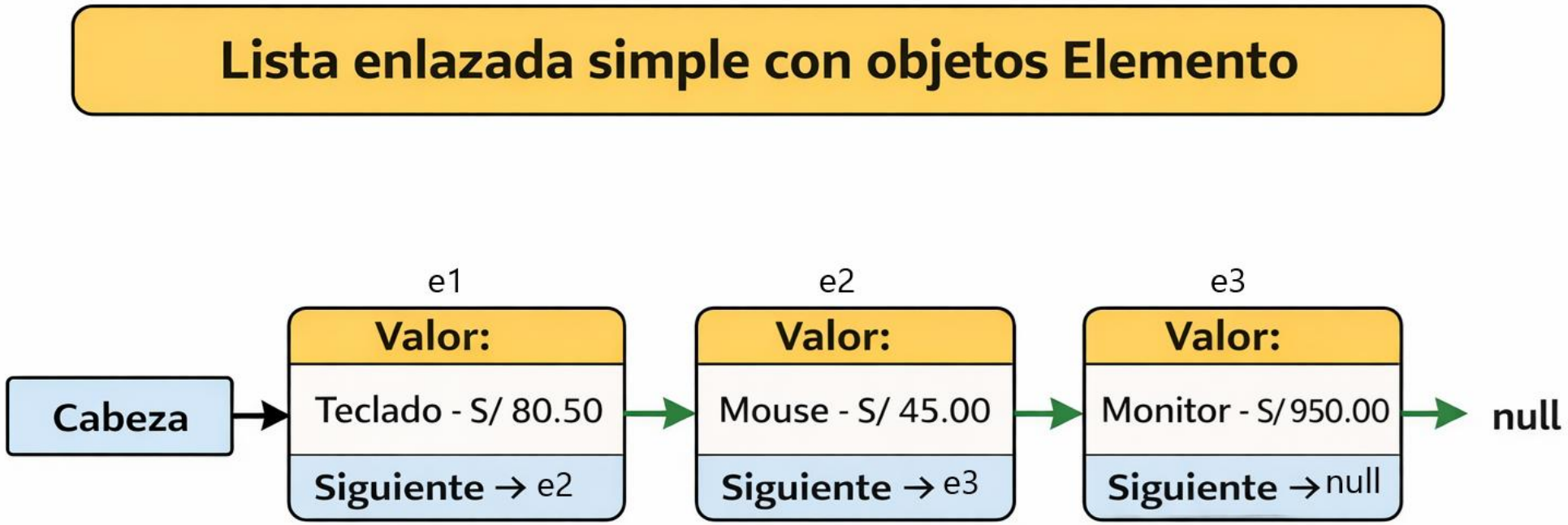
DECLARACIÓN DE UN NODO

```
class Elemento
{
    public string Nombre { get; set; }
    public double Precio { get; set; }

    public Elemento(string nombre, double precio)
    {
        Nombre = nombre;
        Precio = precio;
    }

    public override string ToString()
    {
        return $"{Nombre} - ${Precio}";
    }
}
```

```
class Nodo
{
    public Elemento Dato { get; set; }
    public Nodo Siguiente { get; set; }

    public Nodo(Elemento dato)
    {
        Dato = dato;
        Siguiente = null;
    }
}
```

$e_1, e_2, \ldots e_n$ son valor de tipo `Elemento`.

DECLARACIÓN DE UN NODO



Cada nodo almacena un **objeto Elemento** (Nombre y Precio), y una referencia al **siguiente** nodo.

RECORRIDO DE UNA LISTA ENLAZADA

Algoritmo (La lógica)

1. Empezar en la cabeza.
2. Avanzar nodo por nodo.
3. Detenerse cuando se llegue a null.

Pseudocódigo

```
Inicio
    actual ← cabeza
    Mientras actual ≠ null hacer
        procesar el dato de actual
        actual ← actual.siguiente
    FinMientras
Fin
```

RECORRIDO DE UNA LISTA ENLAZADA

C#

```
public void Recorrer()
{
    Nodo actual = Cabeza;
    while (actual != null)
    {
        Console.WriteLine(actual.Dato);
        actual = actual.Siguiente;
    }
}
```

Interpretación

El método recorre la lista desde la cabeza hasta el final.

En cada iteración muestra el valor del nodo actual y luego avanza al siguiente nodo.

BÚSQUEDA EN UNA LISTA ENLAZADA

Concepto

- La búsqueda en una lista enlazada consiste en recorrer los nodos desde la cabeza hasta encontrar el elemento deseado o llegar al final de la lista.
- Como la lista no tiene acceso aleatorio, la búsqueda se realiza de forma secuencial, nodo por nodo.
- Se compara el dato buscado con el contenido de cada nodo hasta encontrar coincidencia o llegar a null.

Pseudocódigo

```
Inicio
    actual ← cabeza
    Mientras actual ≠ null hacer
        Si actual.Valor es igual al dato buscado entonces
            retornar actual
        FinSi
        actual ← actual.Siguiente
    FinMientras
    retornar null
Fin
```

BÚSQUEDA EN UNA LISTA ENLAZADA

C#

```
public Nodo Buscar(string nombre)
{
  Nodo actual = cabeza;
  while (actual != null)
  {
     if (actual.Dato.Nombre.Equals(nombre,
         StringComparison.OrdinalIgnoreCase))
     {
        return actual; // Encontrado
     }
     actual = actual.Siguiente;
  }
  return null; // No encontrado
}
```

Interpretación

- El método recorre la lista desde la cabeza.
- En cada iteración compara el nombre buscado con el dato almacenado en el nodo.
- Si encuentra coincidencia, retorna el nodo; de lo contrario, continúa hasta llegar a null.

ORDENAMIENTO DE UNA LISTA ENLAZADA SIMPLE

¿Qué significa ordenar una lista enlazada?

Criterio de orden

Se establece un criterio de comparación sobre el campo **Dato** de cada nodo. Por ejemplo, Precio o Nombre del Elemento.

Sin reasignación de memoria

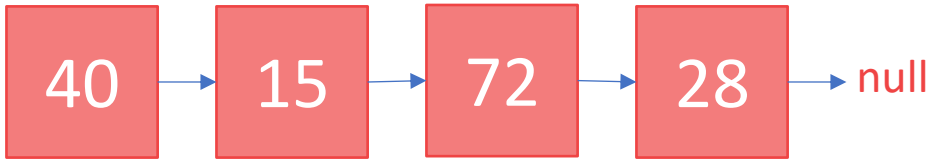
En lugar de mover bloques de memoria, se intercambia el contenido (Dato) de los nodos o se redirigen los punteros **Siguiente**.

Lista vs Arreglo

No existe acceso aleatorio al i-ésimo nodo. Cada acceso requiere un recorrido secuencial, lo que influye en la eficiencia.

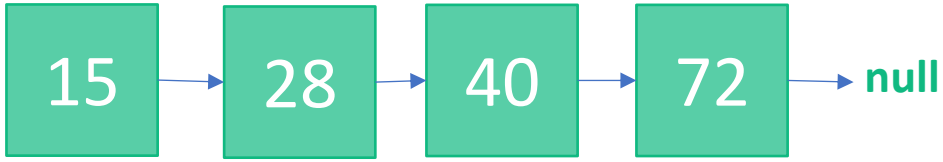
Ejemplo de ordenamiento

ANTES



↓ ordenar por precio ascendente

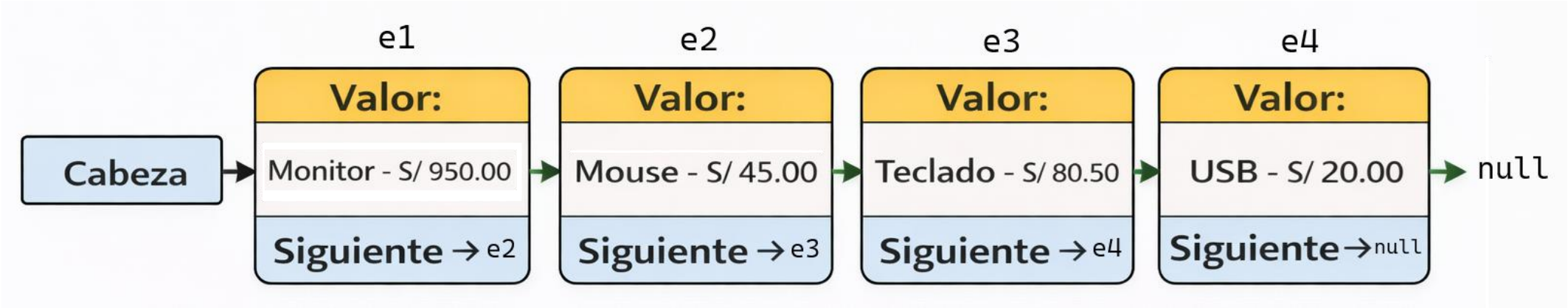
DESPUÉS



ORDENAMIENTO DE UNA LISTA ENLAZADA SIMPLE

Lista inicial	40	15	72	28	null
Pasada 1 → comparar [40,15] → intercambiar	15	40	72	28	null
Pasada 1 → comparar [40,72] → sin cambio	15	40	72	28	null
Pasada 1 → comparar [72,28] → intercambiar	15	40	28	72	null
Pasada 2 → comparar [40,28] → intercambiar	15	28	40	72	null
Pasada 3 → sin intercambios → lista ordenada ✓	15	28	40	72	null

ORDENAMIENTO DE UNA LISTA ENLAZADA SIMPLE



- En este ejemplo, la lista enlazada simple se ordena alfabéticamente por el campo Nombre del objeto Elemento.
- El orden se logra reorganizando los enlaces (Siguiente) entre nodos, sin modificar el contenido de cada nodo.
- Después del ordenamiento, la secuencia queda: Monitor → Mouse → Teclado → USB → null.

ORDENAMIENTO DE UNA LISTA ENLAZADA SIMPLE

Ordenamiento burbuja

1. **Fase externa**

   Se repite el recorrido completo de la lista un máximo de n−1 veces, donde n es la longitud de la lista.

2. **Comparación de pares**

   En cada pasada, se comparan los valores de dos nodos adyacentes: actual y siguiente.

3. **Intercambio de valores**

   Si **Nodo.Dato > Nodo.Siguiente.Dato**, se intercambia el contenido de los nodos, no los punteros.

4. **Terminación anticipada**

   Si en una pasada no ocurre ningún intercambio, la lista ya está ordenada y el algoritmo termina.

C#

```
public void OrdenamientoBurbuja()
{
    if (cabeza == null) return; // Lista vacía
    bool hayIntercambio;
    do
    {
        hayIntercambio = false;
        Nodo actual = cabeza;
        while (actual.Siguiente ≠ null)
        {
            if (actual.Dato.Precio > actual.Siguiente.Dato.Precio)
            {
                // Intercambiar los datos de los nodos
                Elemento temp = actual.Dato;
                actual.Dato = actual.Siguiente.Dato;
                actual.Siguiente.Dato = temp;
                hayIntercambio = true;
            }
            actual = actual.Siguiente;
        } // Buble hile
    } while (hayIntercambio);
}
```

ORDENAMIENTO DE UNA LISTA ENLAZADA SIMPLE

Ordenamiento por Inserción (Insertion Sort)

Analogía con un mazo de cartas

① **Partición implícita**

La lista se divide conceptualmente en dos partes: la sublista izquierda (ya ordenada) y la sublista derecha (pendiente de procesar).

② **Selección de la clave**

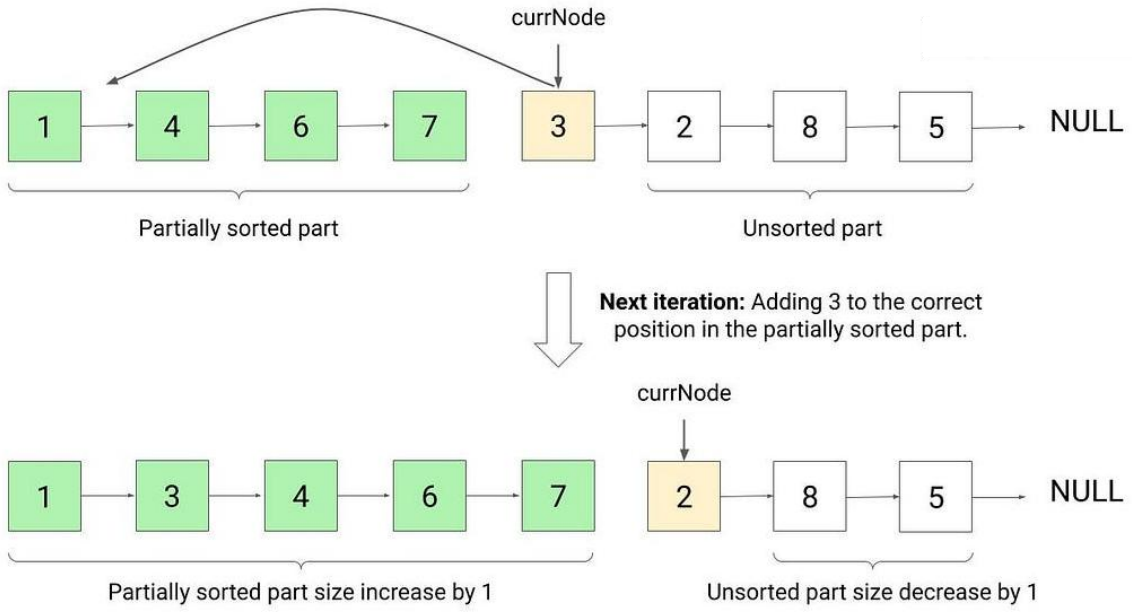
En cada iteración se toma el primer nodo de la parte no ordenada como elemento clave (key).

③ **Búsqueda de posición**

Se recorre la parte ordenada para encontrar la posición correcta donde debe insertarse la clave.

④ **Inserción**

Se redirigen los punteros Siguiente para insertar el nodo en la posición correcta, sin mover bloques de memoria.



ORDENAMIENTO DE UNA LISTA ENLAZADA SIMPLE

Ordenamiento por inserción

1. **Iniciar desde el segundo nodo**

   El primer nodo (cabeza) se considera ordenado. Se comienza a partir del segundo nodo de la lista.

2. **Extraer el nodo clave**

   Se desconecta el nodo actual de la lista. El Dato (Elemento) es la clave que se va a insertar en la posición correcta..

3. **Recorrer la parte ordenada**

   Se recorre desde la cabeza hasta encontrar el nodo anterior al punto de inserción (donde Dato.Precio ≤ clave.Precio).

4. **Reinsertar en posición correcta**

   Se redirigen los punteros Siguiente para colocar el nodo extraído en su posición correcta dentro de la parte ordenada.

C#

```
public void OrdenamientoInsercion()
{
    if (cabeza == null || cabeza.Siguiente == null)
        return; // Lista vacía o con un solo elemento
    Nodo actual = cabeza.Siguiente;
    cabeza.Siguiente = null; // Desconectar el primer nodo
    while (actual ≠ null)
    {
        Nodo siguiente = actual.Siguiente; // Guardar el siguiente nodo
        if (actual.Dato.Precio < cabeza.Dato.Precio)
        {   // Insertar al inicio
            actual.Siguiente = cabeza;
            cabeza = actual;
        } else {// Buscar la posición correcta en la parte ordenada
            Nodo temp = cabeza;
            while (temp.Siguiente ≠ null &&
                   temp.Siguiente.Dato.Precio < actual.Dato.Precio)
            {
                temp = temp.Siguiente;
            }
            // Insertar después de temp
            actual.Siguiente = temp.Siguiente;
            temp.Siguiente = actual;
        }
        actual = siguiente; // Mover al siguiente nodo
    } // Bucle while
}
```

ORDENAMIENTO DE UNA LISTA ENLAZADA SIMPLE

Lista inicial

Monitor S/950

Mouse S/45 — Teclado S/80.50 — USB S/20

Paso 1: clave = Mouse (S/45)

Mouse S/45 — Monitor S/950

S/45 < S/950 → insertar al inicio

Teclado s/80.50 — USB S/20

Paso 2: clave = Teclado (S/80.50)

Mouse S/45 — Teclado S/80.50 — Monitor S/950

S/45 < S/80.50 < S/950 → insertar en medio

USB S/20

Paso 3: clave = USB (S/20)

USB S/20 — Mouse S/45 — Teclado S/80.50 — Monitor S/950

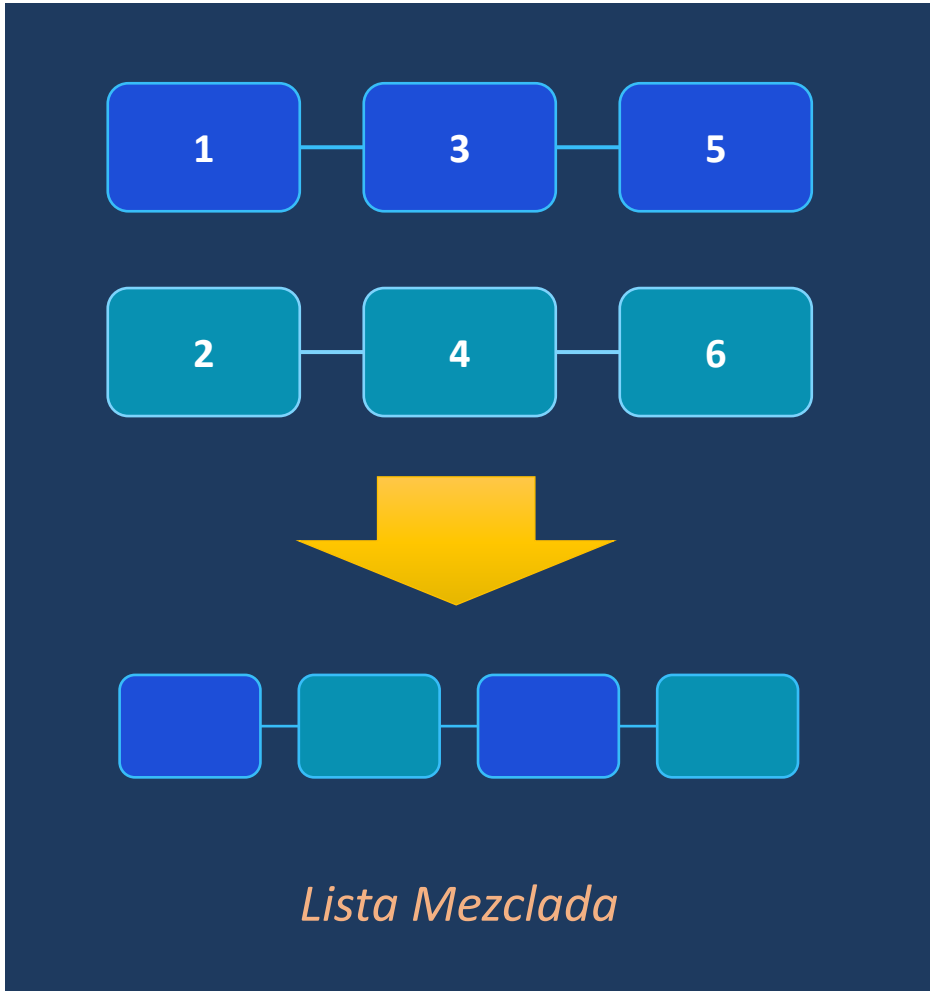
S/20 < S/45 → insertar al inicio ✓ Lista ordenada

MEZCLA DE DOS LISTAS ENLAZADAS SIMPLES

Definición Formal

La mezcla (merge) de dos listas enlazadas simples es una operación que combina los nodos de ambas listas en una nueva lista resultante, siguiendo un criterio predefinido de ordenamiento o simplemente concatenando los elementos.

El proceso requiere el recorrido de ambas listas y la manipulación de los punteros **Siguiente** de los nodos.



Fuente: Cormen, T., Leiserson, C., Rivest, R. & Stein, C. (2022). Introduction to Algorithms, 4th Ed. MIT Press.

MEZCLA DE DOS LISTAS ENLAZADAS SIMPLES

Proceso Visual de Mezcla Ordenada

Lista A (ordenada):

Lista B (ordenada):

Resultado Mezclado:

Fuente: Cormen, T., Leiserson, C., Rivest, R. & Stein, C. (2022). Introduction to Algorithms, 4th Ed. MIT Press.

MEZCLA DE DOS LISTAS ENLAZADAS SIMPLES

Algoritmo de mezcla ordenada

01

Verificar casos base

Si ListaA es null → retornar ListaB. Si ListaB es null → retornar ListaA.

02

Crear nodo centinela (dummy)

Nodo auxiliar para simplificar la lógica. El resultado comienza en dummy.Siguiente.

03

Comparar nodos actuales

Mientras ambos punteros (pA, pB) no sean null: comparar pA.Dato vs pB.Dato.

04

Enlazar el nodo menor

El nodo con menor valor se enlaza al resultado. El puntero de esa lista avanza al siguiente.

05

Conectar la cola restante

Al agotar una lista, se enlaza directamente el resto de la lista no vacía.

06

Retornar resultado

Se retorna dummy.Siguiente como cabeza de la lista mezclada y ordenada.

Fuente: Cormen, T., Leiserson, C., Rivest, R. & Stein, C. (2022). Introduction to Algorithms, 4th Ed. MIT Press.

MEZCLA DE DOS LISTAS ENLAZADAS SIMPLES

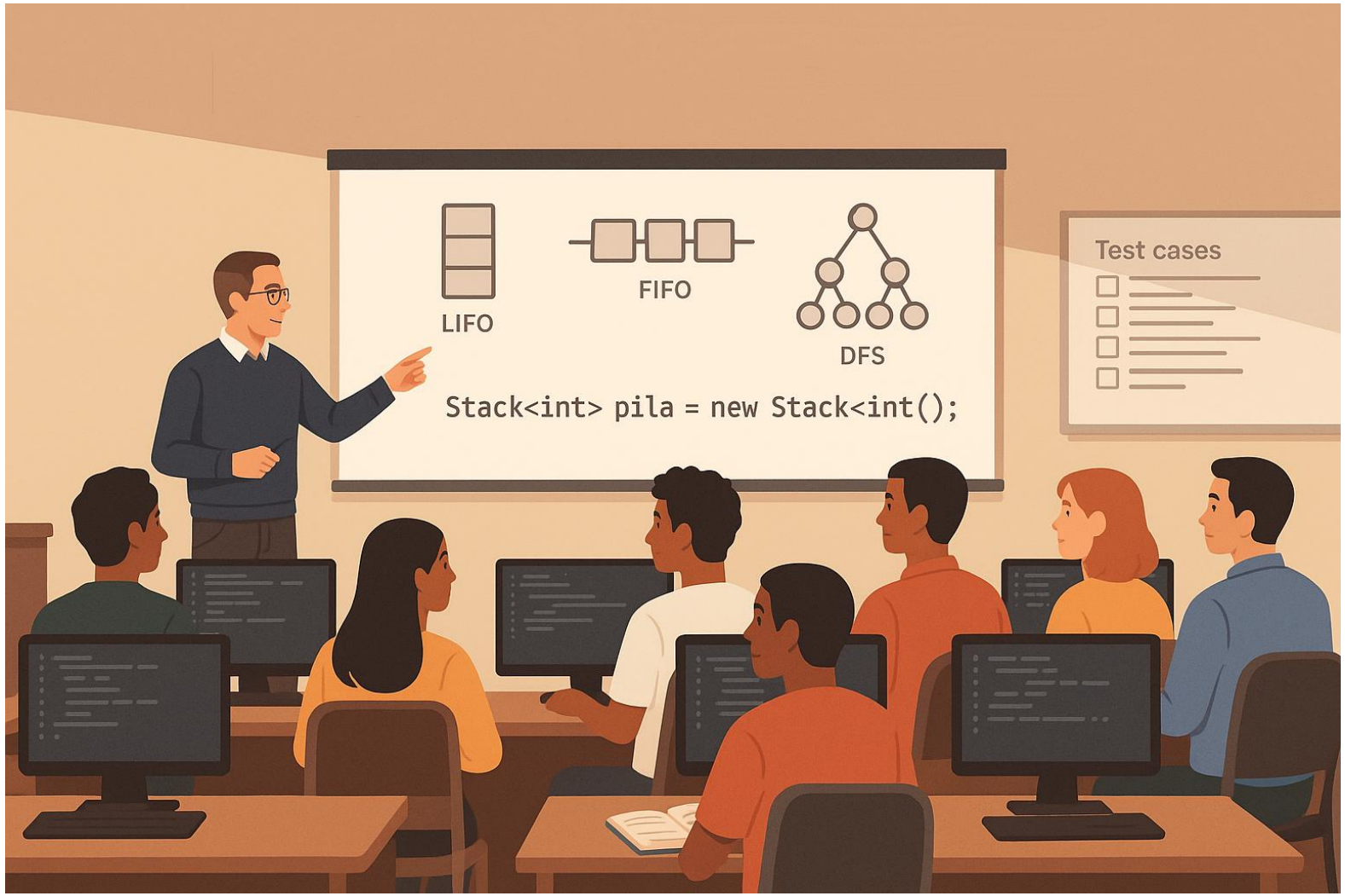
Traza de ejecución

A = [1 → 3 → 5] | B = [2 → 4 → 6]

Paso	pA.Dato	pB.Dato	Acción	Resultado parcial
1	1	2	1 ≤ 2 → tomar de A	1
2	3	2	3 > 2 → tomar de B	1 → 2
3	3	4	3 ≤ 4 → tomar de A	1 → 2 → 3
4	5	4	5 > 4 → tomar de B	1 → 2 → 3 → 4
5	5	6	5 ≤ 6 → tomar de A	1 → 2 → 3 → 4 → 5
6	null	6	A agotada → cola de B	1 → 2 → 3 → 4 → 5 → 6

✅ **Lista Resultante: 1 → 2 → 3 → 4 → 5 → 6 → null**

ACTIVIDAD EN CLASE



PROBLEMA 1

Una empresa de transporte ferroviario necesita gestionar la lista de pasajeros registrados en sus trenes. Cada pasajero tiene los siguientes datos: N° de Boleto, Nombre, Sexo, Destino y Código de Vagón.

Se requiere implementar una solución con Lista Enlazada Simple en C#, utilizando las clases **Pasajero**, **Nodo** y **ListaEnlazada**, que permita realizar las siguientes operaciones:

a. Registrar un nuevo pasajero insertando al final de la lista.

b. Mostrar la lista completa de pasajeros registrados.

c. Buscar un pasajero por Destino, Código de Vagón y N° de Boleto.

d. Eliminar un pasajero por Destino, Código de Vagón y N° de Boleto.

PROBLEMA 2

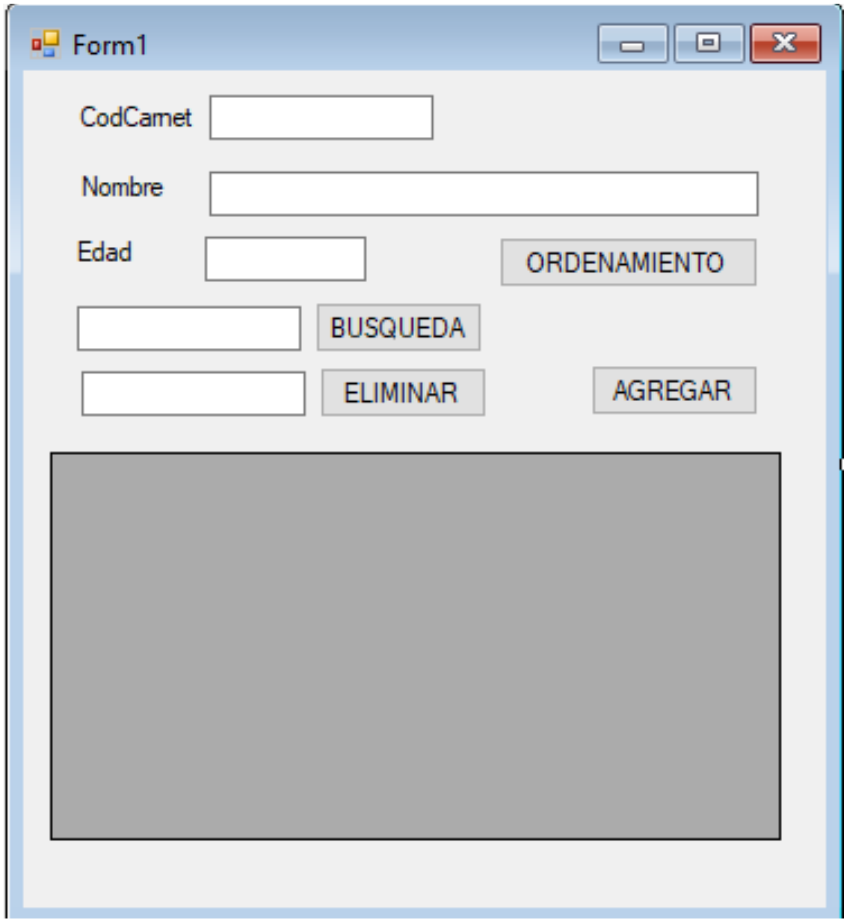
La Universidad Privada del Norte requiere gestionar la lista de alumnos beneficiados con carnets de medio pasaje. Cada alumno tiene los siguientes datos: Cod_Carnet, Nombre y Edad. Se requiere implementar una solución con Lista Enlazada Simple en C#, utilizando las clases **Alumno**, **Nodo** y **ListaEnlazada**, con formulario Windows mostrando los resultados en un **DataGridView**, que permita realizar las siguientes operaciones:

a. Registrar un nuevo alumno insertando al final de la lista y reflejarlo inmediatamente en el **DataGridView**.

b. Buscar un alumno por **Cod_Carnet** y mostrar el resultado en un mensaje flotante.

c. Eliminar un alumno por **Cod_Carnet** y actualizar el **DataGridView**.

d. Ordenar la lista de forma ascendente por **Cod_Carnet** aplicando el algoritmo sobre la lista existente y reflejando el resultado en el **DataGridView**.



PROBLEMA 3

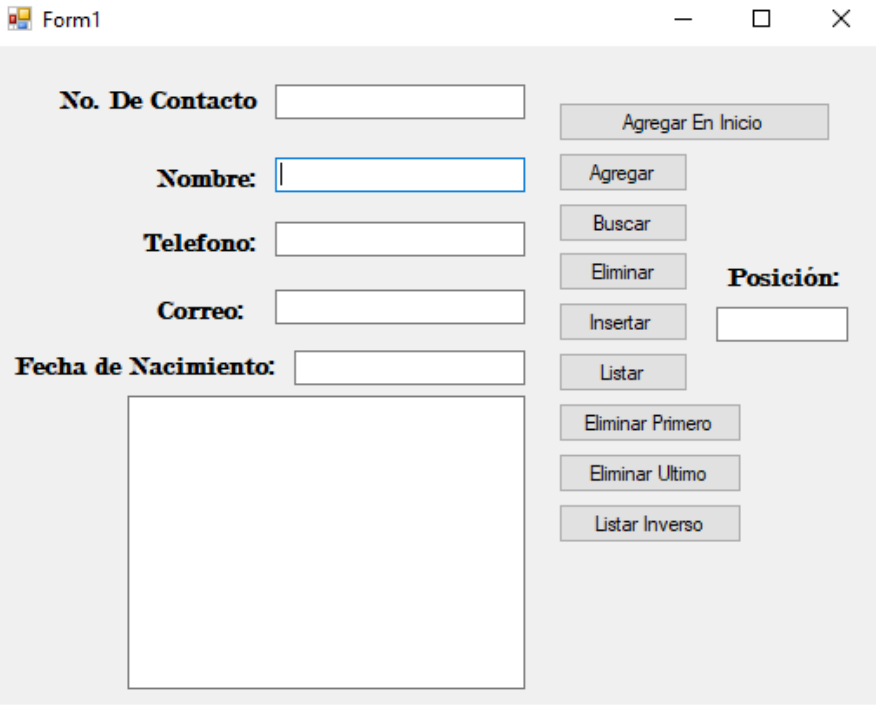
Se realizará un evento por el aniversario la empresa Starbucks. De lo cual requieren sus datos de los clientes. (N°de Contacto, Nombre, Teléfono, Correo y Fecha de Nacimiento) Se necesita los siguientes datos.

a. Insertar al inicio los datos del cliente.

b. Generar una búsqueda por N° de contacto.

c. Eliminar por primero, ultimo de cada cliente.

d. Mostrar la lista de cliente.



CONCLUSIONES

- Las listas simplemente enlazadas permiten un crecimiento dinámico sin requerir memoria contigua, lo que las hace superiores a los arreglos en escenarios de inserciones y eliminaciones frecuentes en posiciones arbitrarias. Sin embargo, cada nodo consume memoria adicional para almacenar el puntero **Siguiente**, y al no garantizar contigüidad en memoria, generan un impacto negativo en la localidad de caché (cache locality), lo que reduce el rendimiento en recorridos secuenciales intensivos comparados con arreglos en hardware moderno.
- La elección entre una lista enlazada simple y un arreglo debe fundamentarse en el análisis de complejidad de las operaciones requeridas por el problema. La lista enlazada simple es la estructura preferible cuando las operaciones dominantes son inserciones y eliminaciones en los extremos, y el acceso aleatorio no es un requerimiento frecuente.

CONCLUSIONES

- La lista enlazada simple constituye el fundamento conceptual y estructural sobre el que se construyen estructuras de mayor complejidad. Las pilas (LIFO) y las colas (FIFO), que se estudiarán en sesiones posteriores, pueden implementarse directamente sobre una lista enlazada simple con operaciones O(1) en sus extremos. Asimismo, las listas de adyacencia para la representación de grafos utilizan este mismo principio. Comprender con precisión el manejo de punteros **Siguiente** en esta sesión es, por tanto, un requisito directo para abordar correctamente dichas estructuras.

- Implementar manualmente una lista enlazada simple a través de las clases **Nodo** y **ListaEnlazada** en C# no es un ejercicio académico aislado, sino el fundamento conceptual que permite comprender cómo se reserva memoria dinámicamente para cada nodo en tiempo de ejecución, cómo los objetos se interconectan mediante referencias, y qué ocurre internamente cuando se inserta o elimina un elemento; comprensión que resulta indispensable para utilizar con criterio y precisión la clase **LinkedList<T>** de la biblioteca estándar de .NET, que implementa una lista doblemente enlazada con operaciones eficientes en ambos extremos, y que un programador sin estos fundamentos utilizaría de forma mecánica, sin capacidad de diagnosticar problemas de rendimiento ni tomar decisiones de diseño justificadas en proyectos reales.

¿PREGUNTAS O COMENTARIOS?

Comprobación del Logro

Cuestionario en la plataforma

Nivel	Rango
Nivel 4	17 – 20
Nivel 3	13 – 16
Nivel 2	9 – 12
Nivel 1	0 - 8

Comprobación del Logro

Criterio	Nivel 4 (Excelente)	Nivel 3 (Bueno)	Nivel 2 (Regular)	Nivel 1 (Deficiente)
Comprensión conceptual	Demuestra comprensión completa de los conceptos clave.	Comprende la mayoría de los conceptos clave.	Muestra comprensión parcial, con algunas confusiones.	No demuestra comprensión clara de los conceptos.
Aplicación de conocimientos	Aplica correctamente los conceptos en ejemplos o escenarios.	Aplica conceptos con algunos errores menores.	Aplica de forma superficial o con errores evidentes.	No logra aplicar los conceptos o lo hace incorrectamente.
Claridad y precisión en respuestas	Las respuestas son claras, bien estructuradas y precisas.	Las respuestas son mayormente claras, con leves imprecisiones.	Respuestas poco claras o con ideas mal organizadas.	Respuestas confusas, incompletas o incoherentes.
Cobertura del contenido esperado	Responde completamente las 5 preguntas, desarrollando bien cada una.	Responde correctamente 4 de las 5 preguntas.	Responde correctamente 2 o 3 preguntas.	Responde solo 1 pregunta correctamente o no responde.

BIBLIOGRAFIA REFERENCIAL

- Ceballos Sierra, F. Microsoft C#: Curso de Programación (2a.ed.) 2014 https://elibronet.eu1.proxy.openathens.net/es/lc/upnorte/titulos/106417
- Cesar Liza Avila; Estructura de datos con C/C++
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley Professional.
- Weiss, M. A. (2012). Data Structures and Algorithm Analysis in C# (3rd ed.). Pearson.
- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Java (6th ed.). Wiley.

GRACIAS

UPN

UNIVERSIDAD
PRIVADA
DEL NORTE